

Лабораторная работа 3. Импорт данных и построение срезов

Теоретическая часть

1. Способы загрузки данных в R

В R предусмотрены несколько способов загрузки данных из различных источников, таких как текстовые файлы, электронные таблицы, базы данных и API. В зависимости от формата данных используются разные функции.

1.1 Чтение данных из CSV-файлов

CSV (Comma-Separated Values) – один из самых распространенных форматов хранения табличных данных. Для его загрузки используется функция `read.csv()`:

```
my_data <- read.csv("data.csv", header = TRUE, sep = ",")
```

Где:

- `header = TRUE` – указывает, что первая строка содержит названия столбцов.
- `sep = ","` – задает разделитель значений.

1.2 Чтение данных из Excel

Для работы с файлами Excel используются пакеты `readxl` и `openxlsx`.

```
library(readxl)  
data <- read_excel("data.xlsx", sheet = 1)
```

Здесь `sheet = 1` указывает номер листа в файле.

1.3 Чтение данных из JSON

Для работы с JSON используется пакет `jsonlite`.

```
library(jsonlite)  
data <- fromJSON("data.json")
```

1.4 Импорт данных из базы данных

Подключение к базам данных выполняется через пакеты `DBI` и `RMySQL`.

```
library(DBI)  
conn <- dbConnect(RMySQL::MySQL(), dbname = "database_name", host =  
"localhost", user = "user", password = "password")
```

```
data <- dbGetQuery(conn, "SELECT * FROM table_name")
dbDisconnect(conn)
```

Эти методы позволяют загружать данные в R из различных источников для дальнейшего анализа.

2. Форматы данных: CSV, Excel, JSON

В R поддерживаются различные форматы данных, которые можно импортировать и экспортировать для анализа. Основные форматы включают:

2.1 CSV (Comma-Separated Values)

CSV – один из самых популярных форматов хранения данных. Каждая строка представляет собой запись, а значения разделяются запятыми или другими символами (такими как ; или).

- Преимущества:
 - Простота использования
 - Совместимость с различными программами (Excel, базы данных, текстовые редакторы)
- Чтение CSV-файлов в R выполняется с помощью `read.csv()`:

```
my_data <- read.csv("data.csv", header = TRUE, sep = ",")
```

- Запись в CSV осуществляется с помощью `write.csv()`:

```
write.csv(my_data, "output.csv", row.names = FALSE)
```

2.2 Excel (XLS/XLSX)

Файлы Excel более удобны для представления данных благодаря поддержке форматирования и формул. Для работы с ними используются пакеты `readxl` и `openxlsx`:

- Чтение данных из Excel-файла:

```
library(readxl)
data <- read_excel("data.xlsx", sheet = 1)
```

- Запись данных в Excel-файл:

```
library(openxlsx)
write.xlsx(data, "output.xlsx")
```

2.3 JSON (JavaScript Object Notation)

JSON – это формат хранения структурированных данных, широко используемый для передачи данных между веб-приложениями и серверами.

- Чтение JSON в R выполняется с помощью пакета `jsonlite`:

```
library(jsonlite)
data <- fromJSON("data.json")
```

- Запись данных в JSON-файл:

```
write_json(data, "output.json")
```

Каждый из этих форматов имеет свои преимущества и применяется в зависимости от целей анализа данных.

3. Фильтрация и подвыборки

Фильтрация данных – это процесс выбора определенных строк или столбцов из набора данных на основе заданных условий. В R этот процесс выполняется с помощью логических выражений и функций для работы с таблицами данных.

3.1 Фильтрация строк по условиям

Для фильтрации строк в `data.frame` используется логическая индексация.

```
# Выбор строк, где значение столбца Age больше 30
data[data$Age > 30, ]
```

Фильтрацию можно комбинировать с операторами `&` (и) и `|` (или):

```
# Выбор строк, где Age больше 30 и Salary превышает 50000
data[data$Age > 30 & data$Salary > 50000, ]
```

3.2 Выбор столбцов

Для выбора конкретных столбцов используется индексация по имени или номеру:

```
# Выбор столбцов Name и Salary
data[, c("Name", "Salary")]
```

3.3 Использование функции `subset()`

Функция `subset()` позволяет фильтровать строки и выбирать столбцы более читаемым способом.

```
# Выбор строк с Age > 30 и только Name и Salary  
subset(data, Age > 30, select = c(Name, Salary))
```

3.4 Использование `dplyr` для фильтрации

Пакет `dplyr` предлагает удобный синтаксис для работы с данными.

```
library(dplyr)  
data_filtered <- data %>% filter(Age > 30) %>% select(Name, Salary)
```

Фильтрация и подвыборки помогают работать с нужными данными и исключать ненужные записи для последующего анализа.

Практическая часть

1. Импортировать данные из CSV-файла.
2. Сделать подвыборку данных по критериям.
3. Экспортировать данные в файл.

Контрольные вопросы

1. Какие функции используются для загрузки CSV-файлов?
 2. Как выбрать определенные строки или столбцы из таблицы?
 3. Как фильтровать данные по условиям?
 4. Как сохранить обработанные данные в файл?
 5. Чем отличается `read.csv` от `read.table`?
-